REM Offline Engineering Notes

MyEmulator2 project

REM offline engineering index

Use this as the short map for the curated phone bundle.

Need	Document
learn the platform and build software	`Inside_REM_Edition3.pdf`
current target snapshot	`REM_CURRENT_TARGET_MANIFEST.md`
syscall numbers and implementation status	`REM_CURRENT_SYSCALL_REFERENCE.md`
write syscall wrappers in assembly	`REM_ASSEMBLY_SYSCALL_COOKBOOK.md`
understand the static userspace layout	`REM_LINKER_SCRIPT_GUIDE.md`
make a bare-metal image	`REM_BARE_METAL_IMAGE_RECIPE.md`
know the terminal boundary	`REM_TERMINAL_CAPABILITY_MATRIX.md`
find executable fixtures	`REM_KNOWN_GOOD_FIXTURES.md`
prepare the phone bundle	`REM_OFFLINE_BUILD_MANIFEST.md`
verify the bundle	`tools/check-offline-flight-kit.sh`

The full source repository remains the workstation source of truth. These documents are the small, stable layer to carry with the PDFs.

REM current target manifest

This manifest freezes the target described by the accompanying offline documents. It is a snapshot, not a promise that later repository revisions retain every interface.

Item	Snapshot
CPU/QEMU target	MyEmulator2 / `myemulator32`
Source revision	`d463af9f6298446ca94ef0710efb42ee21ac2f7c` plus the current working-tree changes
Object format	ELF32 little-endian
Assembler/linker	`.toolchain-install/bin/myemulator2-elf-as`, `myemulator2-elf-ld`
QEMU	`.qemu-build/qemu-system-myemulator32`
Kernel image	`.linux-build/build/vmlinux`
Userspace linker script	`toolchain/userspace/myemulator2-user.ld`
Static musl prefix	`.musl-install` (optional; absent in this snapshot)

The local musl build was attempted from the cached musl 1.2.5 source and stopped in `src/stdio/vfwscanf.c`: the current target compiler treats its `wchar_t` pointer-sign warning as an error. No installed sysroot is claimed until that toolchain warning is resolved and a hello program is linked.

The source tree is the authority for this snapshot. The phone bundle need not contain the whole repository: copy the PDFs, the source paths named in `REM_OFFLINE_BUILD_MANIFEST.md`, and any binaries listed as present by the pre-flight checker.

Implemented syscall dispatch and regression coverage are separate facts. Use `tools/generate-myemulator2-syscall-referen` to regenerate the syscall table from `linux/arch/myemulator2/kernel/syscall.c` after changing the tree.

Current MyEmulator2 syscall reference

Generated from `linux/arch/myemulator2/kernel/syscall.c`. This records implemented dispatcher cases; it is not a claim of complete Linux ABI coverage. Tested coverage is a separate fact and is recorded by the named regression scripts.

The userspace convention is `r1` = number/result and `r2`–`r7` = arguments. The `syscall` immediate is a diagnostic field; current Linux handling ignores it, so examples use `syscall 0`.

Number	Name	Kernel status
17	`getcwd`	dispatcher case
21	`access`	dispatcher case
23	`dup`	dispatcher case
24	`dup3`	dispatcher case
25	`fcntl`	dispatcher case
29	`ioctl`	dispatcher case
34	`mkdirat`	dispatcher case
35	`unlinkat`	dispatcher case
38	`renameat`	dispatcher case
46	`ftruncate`	dispatcher case
49	`chdir`	dispatcher case
50	`fchdir`	dispatcher case
52	`fchmod`	dispatcher case
53	`fchmodat`	dispatcher case
56	`openat`	dispatcher case
57	`close`	dispatcher case
59	`pipe2`	dispatcher case
61	`getdents64`	dispatcher case
62	`lseek`	dispatcher case
63	`read`	dispatcher case
64	`write`	dispatcher case
65	`readv`	dispatcher case
66	`writev`	dispatcher case
72	`pselect6`	dispatcher case
73	`ppoll`	dispatcher case
74	`fsync`	dispatcher case
78	`readlinkat`	dispatcher case
79	`fstatat`	dispatcher case
80	`fstat`	dispatcher case
81	`sync`	dispatcher case
83	`fdatasync`	dispatcher case
88	`utimensat`	dispatcher case
93	`exit`	dispatcher case
94	`exit_group`	dispatcher case
96	`set_tid_address`	dispatcher case
99	`set_robust_list`	dispatcher case
101	`nanosleep`	dispatcher case
124	`sched_yield`	dispatcher case
129	`kill`	dispatcher case
130	`tkill`	dispatcher case
131	`tgkill`	dispatcher case
132	`sigaltstack`	dispatcher case
134	`rt_sigaction`	dispatcher case
135	`rt_sigprocmask`	dispatcher case
144	`setgid`	dispatcher case
146	`setuid`	dispatcher case

Number	Name	Kernel status
154	`setpgid`	dispatcher case
155	`getpgid`	dispatcher case
156	`getsid`	dispatcher case
157	`setsid`	dispatcher case
160	`uname`	dispatcher case
166	`umask`	dispatcher case
172	`getpid`	dispatcher case
173	`getppid`	dispatcher case
174	`getuid`	dispatcher case
175	`geteuid`	dispatcher case
176	`getgid`	dispatcher case
177	`getegid`	dispatcher case
178	`gettid`	dispatcher case
214	`brk`	dispatcher case
215	`munmap`	dispatcher case
220	`clone`	dispatcher case
221	`execve`	dispatcher case
222	`mmap`	dispatcher case
226	`mprotect`	dispatcher case
260	`wait4`	dispatcher case
261	`prlimit64`	dispatcher case
278	`getrandom`	dispatcher case
291	`statx`	dispatcher case
403	`clock_gettime64`	dispatcher case
422	`futex`	dispatcher case

Regenerate with:

```
python3 tools/generate-myemulator2-syscall-reference.py
```

Assembly syscall cookbook

All examples use the current MyEmulator2 GAS convention: put the syscall number in `r1`, arguments in `r2`–`r7`, execute `syscall 0`, and find the result in `r1`. A negative result is a kernel `-errno`; preserve any live caller state before making the call. Numbers are generated in `REM_CURRENT_SYSCALL_REFERENCE.md`.

Scalar call: close or exit

```
li   r1, 57          # close
li   r2, 3           # fd
syscall 0
bltz r1, .error
```

For process termination use number 94 (`exit_group`) and status in `r2`.

Pointer and length: write/read

```
li   r1, 64          # write
li   r2, 1           # fd
la   r3, message
li   r4, message_end-message
syscall 0
```

`read` is number 63 with the same fd, buffer, count pattern. Reads and writes may be short; loop on the remaining count. Retry on `-EINTR`, and only retry `-EAGAIN` when the file descriptor is intentionally nonblocking.

Pathname: openat

Number 56 takes directory fd, pathname pointer, flags and mode in `r2`–`r5`. The pathname must be NUL-terminated and remain valid for the call. Check the negative result before using it as a file descriptor.

Pointer to a structure: ioctl

Number 29 takes fd, request number and structure pointer. Structure layout and request constants belong to the target headers; do not guess offsets in an assembly-only program. This is why the worked terminal bridge keeps termios layout in a small C helper.

Waiting: ppoll

Number 73 takes a `struct pollfd` array pointer, count, timeout pointer and signal-mask pointer. The pointed-to structures must use the target's 32-bit layout. See the POSIX/Linux reference and headers before writing them by hand.

Processes: clone, execve, wait4

The dispatcher exposes clone (220), execve (221), and wait4 (260). These are five-argument Linux interfaces and are best wrapped in a small assembly module with a documented stack/child-entry contract. Do not substitute a normal `fork` example: `fork` is not the interface exposed by this target.

Memory: brk and mmap

`brk` is 214 and `mmap` is 222. Both return an address or a negative errno; page alignment, flags and protection values come from the target headers. Static musl/minilibc may choose different allocation paths, so inspect the current libc source before replacing its allocator.

MyEmulator2 userspace linker script

The supported static userspace link uses `toolchain/userspace/myemulator2-user.ld`. It sets `_start` as the entry, emits ELF32 little-endian MyEmulator2 output, and places the first loadable segment at `0x02000000 + SIZEOF_HEADERS`. Including `FILEHDR PHDRS` makes the program-header table available to musl startup through `AT_PHDR`.

The script keeps text, read-only data, TLS, initialisation arrays, data, GOT and BSS in the single initial load segment, and emits a `PT_TLS` and `PT_GNU_STACK` header. This is deliberately a small static-link layout, not a dynamic-loader contract. The linker may warn that the load segment is RWX; that warning reflects the current compact script and should not be silently treated as a hardened production layout.

Inspect a linked image with:

```
myemulator2-elf-readelf -l -h hello
myemulator2-elf-objdump -h -d hello
```

Do not replace the script with `-Ttext` merely to make an address look tidy: the ELF headers must remain covered by a load segment for the current musl startup path.

Bare-metal image recipe

Bare-metal ELF and raw firmware are different deliverables. ELF retains symbols and program headers for loaders and inspection; a raw image contains bytes at the platform's reset-image locations. Start with the repository fixture and inspect it before changing the image format:

```
.toolchain-install/bin/myemulator2-elf-as \
  toolchain/examples/minimal.s -o /tmp/minimal.o
.toolchain-install/bin/myemulator2-elf-ld \
  /tmp/minimal.o -o /tmp/minimal.elf
.toolchain-install/bin/myemulator2-elf-readelf -h -l /tmp/minimal.elf
.toolchain-install/bin/myemulator2-elf-objcopy -O binary \
  /tmp/minimal.elf /tmp/minimal.bin
```

The reset/firmware loader used by a particular QEMU test supplies the image at the platform-defined reset locations. Use the named test script or boot recipe for that machine; do not infer a Linux userspace load address from a bare-metal raw image. `tests/programs/README.md` and `tests/run-assembler-qemu-test.sh` are the current fixture-oriented entry points.

Terminal capability boundary

This is a boundary map for offline development. "Documented" means the interface is described in the book or references; "exercised" means a named fixture or regression uses it. It does not claim a complete terminal emulator.

Capability	Status	Evidence/boundary
byte input/output	documented, exercised	syscall and console fixtures
ANSI/CSI cursor and erase sequences	documented	`Inside_REM_Edition3.pdf`, Appendix E
SGR/reset and cursor visibility	documented	Appendix E
canonical/raw termios concepts	documented	Appendix R and worked terminal program
`VMIN/VTIME`	documented	Appendix R; verify libc headers for values
`TIOCGWINSZ`	documented	Appendix R; wrapper availability is target-dependent
resize signals	conceptual	signal support is documented; interactive delivery needs target verification
full VT220/ECMA-48 behaviour	outside scope	use `VT220_Programmer_Reference_Manual.pdf`

Keep terminal protocol parsing in the terminal module. The editor should consume events, not depend on escape-sequence spelling.

Known-good fixtures and their purpose

Fixture	Demonstrates	Check
`toolchain/examples/minimal.s`	current GAS syntax, link, halt	`tests/run-assembler-qemu-test.sh`
`toolchain/examples/linux-echo.S`	Linux syscall entry and console I/O	userspace regression scripts
`toolchain/examples/linux-minilibc.s`	hosted C/minilibc smoke path	`test-linux-minilibc.py`
`toolchain/examples/linux-tty-probe.S`	tty probing from assembly	inspect/target-specific run
`toolchain/examples/linux-clone-wait.S`	process ABI exercise	userspace examples
`toolchain/examples/multi-main.s` + `multi-functions.s`	multi-object link	worked multi-file example
`tests/programs/*.s`	instruction/firmware fixtures	corresponding `tests/run-*` scripts

An example is not marked tested merely because it exists. Use the named script, record the revision, and preserve the output when preparing an offline bundle.

Curated offline build manifest

The phone bundle is intentionally curated rather than a copy of the whole repository. Keep the complete source tree on the workstation and copy these groups when offline rebuild or inspection is required.

Reading: `Inside_REM_Edition3.pdf`, the assembly manual and cheat sheets, `REM_PROGRAMMERS_GUIDE.pdf`, `REM_POSIX_Linux_Reference.pdf`, `REM_GAS_Assembler_Reference.pdf`, `REM_GNU_LD_Linker_Reference.pdf`, `REM_ELF_Generic_ABI.pdf`, `REM_QEMU_Debugging_Reference.pdf`, `REM_Flight_Cheat_Sheets.pdf`, and the VT220 reference.

Source contracts: `docs/*.md` named by the PDFs, `toolchain/scripts/`, `toolchain/userspace/`, `toolchain/examples/`, `linux/arch/myemulator2/`, `qemu/target/myemulator32/`, `toolchain/userspace/myemulator2-use` and `tests/programs/`.

Optional prebuilt artifacts: `.toolchain-install/bin/`, `.qemu-build/qemu-system-myemulator32`, `.linux-build/build/vmlinux`, and `.musl-install/` when present. These can be omitted if storage is tight and the source/build environment is already available elsewhere.

For the assembly-only flight profile, `.musl-install/` is unnecessary: GAS, the linker, QEMU, the kernel and the assembly fixtures are the required runtime path. Include the musl prefix only if hosted C programs are part of the trip; it is not present in the current snapshot.

Run `tools/check-offline-flight-kit.sh` against the assembled bundle before leaving. It reports missing optional binaries separately from missing reading and source contracts.